



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
مدل‌های مولد عمیق

تمرین چهارم

نام و نام خانوادگی	سید محمد جزایری
شماره دانشجویی	810101399
تاریخ ارسال گزارش	22/11/1404

فهرست مطالب

سؤال اول.....4

4.....	بخش اول
4.....	سوال اول)
4.....	سوال دوم)
5.....	سوال سوم)
6.....	سوال چهارم)
7.....	سوال پنجم)
8.....	بخش دوم
8.....	زیر بخش اول)
8.....	مرحله اول)
9.....	مرحله دوم)
10.....	مرحله سوم)
11.....	مرحله چهارم)
14.....	زیر بخش دوم)
14.....	مرحله اول)
15.....	مرحله دوم)
16.....	مرحله سوم)
17.....	مرحله چهارم)
18.....	مرحله پنجم)

22..... سؤال دوم

22.....	بخش اول
22.....	سوال اول)
23.....	سوال دوم)
25.....	بخش دوم
25.....	مرحله اول)
25.....	مرحله دوم)
26.....	مرحله سوم)
27.....	مرحله چهارم)
29.....	مرحله پنجم)
30.....	مرحله ششم)
32.....	مرحله هفتم)
33.....	مرحله هشتم)

35.....مراجع

فهرست تصاویر

5.....	عکس 1. فرمول زیان DDPM.....
7.....	عکس 2. فرمول DDIM.....
8.....	عکس 3. مقدار دهی بتا.....
9.....	عکس 4. کد تابع perturb.....
10.....	عکس 5. حلقه آموزش.....
11.....	عکس 6. پیاده سازی نمونه برداری DDPM.....
12.....	عکس 7. پیاده سازی نمونه برداری DDIM.....
12.....	عکس 8. 50 نمونه تولید شده توسط DDPM.....
13.....	عکس 9. 50 نمونه تولید شده توسط DDIM.....
14.....	عکس 10. مقایسه زمان اجرا.....
14.....	عکس 11. مقایسه FID score.....
14.....	عکس 12. پرامپت های انتخابی.....
15.....	عکس 13. پنج عکس انتخابی.....
16.....	عکس 14. متد getitem.....
16.....	عکس 15. تولید تصاویر عمومی.....
18.....	عکس 16. حلقه آموزش.....
18.....	عکس 17. عکس تولیدی اول.....
19.....	عکس 18. عکس تولیدی دوم.....
20.....	عکس 19. عکس تولیدی سوم.....
21.....	عکس 20. عکس تولیدی چهارم.....
22.....	عکس 21. عکس تولیدی پنجم.....
25.....	عکس 22. زیان آموزش و تست.....
26.....	عکس 23. منطق تولید نمونه.....
27.....	عکس 24. نمونه واقعی.....
28.....	عکس 25. نمونه تولید شده توسط مدل.....
30.....	عکس 26. هیستوگرام log-returns.....
31.....	عکس 27. KDE of log-returns.....
32.....	عکس 28. مقایسه آماری توزیع مدل و واقعی.....

سؤال اول

بخش اول

سوال اول

در ابتدا متغیر x_t را بگونه‌ای دیگر می نویسیم.

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t \quad \epsilon_t \sim N(0, I)$$

همچنین باید دقت کنیم که $Var(aX) = a^2 Var(X)$ به همین دلیل واریانس عبارت بالا 1 است.

$$x_1 = \sqrt{\alpha_1} x_0 + \sqrt{1 - \alpha_1} \epsilon_1$$

$$x_2 = \sqrt{\alpha_2} x_1 + \sqrt{1 - \alpha_2} \epsilon_2 = \sqrt{\alpha_2} (\sqrt{\alpha_1} x_0 + \sqrt{1 - \alpha_1} \epsilon_1) + \sqrt{1 - \alpha_2} \epsilon_2 = \sqrt{\alpha_2 \alpha_1} x_0$$

$$+ \sqrt{\alpha_2 - \alpha_1 \alpha_2} \epsilon_1 + \sqrt{1 - \alpha_2} \epsilon_2$$

کل دو عبارت خط بالا را یک متغیر تصادفی نرمال با واریانس $1 - \alpha_1 \alpha_2$ می گیریم.

$$Var(\sqrt{\alpha_2 - \alpha_1 \alpha_2} \epsilon_1 + \sqrt{1 - \alpha_2} \epsilon_2) = \alpha_2 - \alpha_1 \alpha_2 + 1 - \alpha_2 = 1 - \alpha_1 \alpha_2$$

$$x_2 = \sqrt{\alpha_2 \alpha_1} x_0 + \sqrt{1 - \alpha_1 \alpha_2} \epsilon_2$$

با استقرا حکم را ثابت می کنیم فرض کنیم تا $t-1$ درست باشد برای t اثبات می کنیم.

$$x_{t-1} = \sqrt{\alpha_{t-1}} x_0 + \sqrt{1 - \alpha_{t-1}} \epsilon_{t-1}$$

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_t = \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} x_0 + \sqrt{1 - \alpha_{t-1}} \epsilon_{t-1}) + \sqrt{1 - \alpha_t} \epsilon_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon_t$$

سوال دوم

مزیت (۱) هدف یادگیری ساده‌تر و پایدارتر (Loss بهتر شرطی می‌شود)

وقتی ϵ را پیش‌بینی می‌کنیم، هدف شبکه در همه‌ی زمان‌ها تقریباً هم‌مقیاس و با توزیع استاندارد است (نویز گوسی با واریانس 1). بنابراین: گرادین‌ها پایدارتر می‌شوند (کمتر دچار انفجار/نابودی). شبکه لازم نیست هم‌زمان با تغییر t ، مقیاس خروجی را مدام تنظیم کند.

مزیت ۲) هم‌ارزی با پیش‌بینی x_0 و μ_θ (ولی با پیاده‌سازی راحت‌تر)

از رابطه‌ی فرم بسته‌ی نویزی‌کردن که در سوال قبل اثبات کردیم میدانیم که اگر شبکه ϵ را بدهد، می‌توانیم خیلی راحت x_0 را تخمین بزنیم (و بعد μ_θ را هم بسازیم). یعنی با یک خروجی ساده (ϵ) عملاً به همان اطلاعاتی می‌رسیم که برای بازسازی/نمونه‌برداری لازم داریم. پس لازم نیست شبکه مستقیماً x_0 یا μ_θ را با مقیاس‌های وابسته به t تولید کند. فرمول‌بندی آموزش اغلب به یک MSE ساده روی ϵ ختم می‌شود که استاندارد و کارا است.

سوال سوم

ایده‌ی اصلی VLB این است که فرآیند معکوس مدل را به پسین واقعی فرآیند روبه‌جلو نزدیک کند. در DDPM، فرآیند روبه‌جلو (نویز اضافه کردن) با q و فرآیند روبه‌عقب (حذف نویز) با P_θ مدل می‌شود. در تجزیه‌ی استاندارد VLB معمولاً چیزی شبیه این داریم:

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t>1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))}_{L_{t-1}} \underbrace{- \log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)}_{L_0} \right]$$

عکس 1. فرمول زیان DDPM

الف) تعبیر مفهومی هر بخش:

1. KL‌های میانی (از $t=2$ تا T)

این‌ها تشویق می‌کنند که در هر گام زمانی، توزیع معکوس مدل

$$P_\theta(x_{t-1} | x_t)$$

به توزیع «ایده‌آل/واقعی» که از فرآیند روبه‌جلو به دست می‌آید نزدیک شود:

$$q(x_{t-1} | x_0, x_t)$$

یعنی مدل یاد بگیرد «اگر این مقدار نویز در x_t دیده شد، بهترین توزیع برای یک قدم عقب‌تر چیست».

2. ترم KL انتهایی (prior matching در T)

تشویق می‌کند که در انتهای فرآیند روبه‌جلو، توزیع x_T شبیه یک prior ساده مثل گاوسی استاندارد شود:

$$q(x_T|x_0) \approx N(0, I)$$

که باعث می‌شود نمونه‌برداری از نویز خام به‌عنوان نقطه شروع ممکن و درست باشد.

3. ترم بازسازی - $\log p_\theta(x_0|x_1)$

تشویق می‌کند وقتی تقریباً به داده نزدیک شدیم (در x_1)، مدل بتواند داده‌ی اصلی x_0 را خوب بازسازی کند.

ب) ایده‌ی اصلی این است: در DDPM اگر مدل را طوری پارامتردهی کنیم که نویز ϵ را پیش‌بینی کند، آن وقت ترم‌های KL وسط ELBO (که فاصله‌ی دو گاوسی هستند) از نظر ریاضی به چیزی تبدیل می‌شوند که عملاً همان خطای مربعی پیش‌بینی نویز است (گاهی با یک وزن وابسته به t). بنابراین به‌جای اینکه همه‌ی KL ها را مستقیم حساب کنیم، می‌توانیم معادل ساده‌ترشان یعنی MSE روی ϵ را کمینه کنیم و تقریباً همان فشار آموزشی را به مدل وارد کنیم. ترم اول صرفاً می‌گوید که داده آخر در مرحله رو به جلو از توزیع گاوسی بیاید که با نویزی کردن مرحله به مرحله محقق می‌شود. ترم سوم هم سعی می‌کند که داده اصلی را به یک مرحله نویزی آن نزدیک کند و چونکه نویز در هر مرحله خیلی کوچک است تقریباً قابل صرف نظر کردن است.

سوال چهارم

الف) مدل DDIM از همان مسیر رو به جلو DDPM استفاده می‌کند ولی در مسیر بازگشت از فرمولی دیگر استفاده می‌کند.

$$x_{t-1} = \underbrace{\sqrt{\alpha_{t-1}} \left(\frac{x_t - \sqrt{1 - \alpha_t} \epsilon_\theta^{(t)}(x_t)}{\sqrt{\alpha_t}} \right)}_{\text{"predicted } x_0"} + \underbrace{\sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \epsilon_\theta^{(t)}(x_t)}_{\text{"direction pointing to } x_t"} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$

عکس 2. فرمول DDIM

پارامتر η پارامتر σ را کنترل می کند و با یکدیگر هم صفر می شوند اگر در فرمول بالا $\sigma_t = 0$ فرمول دیگر هیچ پارامتر تصادفی نخواهد داشت و فقط در جهت محاسبه شده حرکت می کند. (ب) چون DDIM بر خلاف DDPM از یک زنجیره مارکوف غیر قطعی استفاده نمی کند و بعضی گام ها را می پرد در حالی که کیفیت خیلی افت نمی کند. همچنین نویز هایی که DDPM در هر مرحله تزریق می کند مسیر بازگشت به نمونه واقعی را سخت تر می کند و باید آن نویز را از بین برد و این عمل به گام های بیشتری نیاز دارد ولی DDIM وزن این نویزها را کمتر می کند و در نتیجه گام هایش هم کمتر می شوند.

سوال پنجم

الف) در Classifier-Free Guidance یک مدل دیفیوژن واحد آموزش می دهیم که بتواند در دو حالت کار کند:

1. **حالت شرطی** (با شرط c ، مثل متن یا برچسب کلاس)

$$\epsilon_{\theta}(x_t, t, c)$$

2. **حالت بدون شرط** (شرط را حذف می کنیم؛ گاهی $\emptyset = c$ می نویسند)

$$\epsilon_{\theta}(x_t, t, \emptyset)$$

در زمان آموزش، با یک احتمال مشخص، شرط را تصادفی حذف می کنیم تا همان شبکه هم حالت شرطی و هم حالت بدون شرط را یاد بگیرد.

در زمان نمونه برداری، این دو پیش بینی را ترکیب می کنیم تا «راهنمایی» ایجاد شود:

$$\epsilon_{guided} = \epsilon_{\theta}(x_t, t, \emptyset) + s(\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \emptyset))$$

که در آن s همان ضریب راهنمایی (guidance scale) است.

به آن classifier-free می گویند چون هیچ دسته بند جداگانه ای آموزش نمی دهیم یا استفاده نمی کنیم (برخلاف classifier guidance). سیگنال راهنمایی فقط از اختلاف پیش بینی شرطی و بدون شرط خود مدل به دست می آید.

ب) اگر ضریب راهنمایی s را زیاد کنیم:

- کیفیت/همخوانی با شرط بیشتر می‌شود: نمونه‌ها بهتر با متن/برچسب هماهنگ می‌شوند، چون آپدیت‌ها قوی‌تر در جهتی اعمال می‌شوند که تفاوت حالت شرطی و بدون شرط را نشان می‌دهد.
 - تنوع کمتر می‌شود: خروجی‌ها شبیه‌تر و کم‌تنوع‌تر می‌شوند، چون راهنمایی قوی مدل را به سمت مجموعه محدودتری از حالت‌ها که شرط را بهتر ارضا می‌کنند سوق می‌دهد.
- پس افزایش s معمولاً «وفاداری به شرط» را بالا می‌برد ولی تنوع را کاهش می‌دهد (و اگر خیلی بزرگ شود، ممکن است آرتیفکت ایجاد کند یا خروجی‌ها بیش از حد محدود شوند).

بخش دوم

زیر بخش اول

مرحله اول

```
# Create Linear Scheduler - beta1 and beta2 are given in the hyperparameters block
# We precompute useful constants for the forward and reverse diffusion processes.

# betas: (T,)
b_t = torch.linspace(beta1, beta2, timesteps, device=device)

# alphas and cumulative products
a_t = 1.0 - b_t
ab_t = torch.cumprod(a_t, dim=0) # alpha_bar_t
sqrt_ab_t = torch.sqrt(ab_t)
sqrt_one_minus_ab_t = torch.sqrt(1.0 - ab_t)

# A small helper to gather per-timestep scalars for a batch and reshape for broadcasting
def extract(v, t, x_shape):
    t = t.long()
    out = v.gather(0, t)
    return out.view((t.shape[0],) + (1,) * (len(x_shape) - 1))

# posterior variance used in DDPM sampling (q(x_{t-1} | x_t, x_0))
ab_t_prev = torch.cat([torch.tensor([1.0], device=device), ab_t[:-1]], dim=0)
posterior_var = b_t * (1.0 - ab_t_prev) / (1.0 - ab_t)
posterior_log_var_clipped = torch.log(torch.clamp(posterior_var, min=1e-20))
```

عکس 3. مقدار دهی بتا


```
# helper function: perturbs an image to a specified noise level

def perturb_input(x0: torch.Tensor, t: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
    """Sample  $x_t$  from  $q(x_t | x_0)$  at arbitrary timestep(s)  $t$ .

    Args:
        x0: clean images, shape [B,C,H,W]
        t: timesteps, shape [B] (values in [0, T-1])

    Returns:
        x_t: noised images at timestep  $t$ 
        eps: the Gaussian noise used to create  $x_t$ 
    """
    eps = torch.randn_like(x0)
    sqrt_ab = extract(sqrt_ab_t, t, x0.shape)
    sqrt_omab = extract(sqrt_one_minus_ab_t, t, x0.shape)
    x_t = sqrt_ab * x0 + sqrt_omab * eps
    return x_t, eps
```

عکس 4. کد تابع perturb

```

for ep in range(n_epoch):
    print(f"epoch {ep}")
    pbar = tqdm(train_dataloader)

    running = 0.0
    n_steps = 0

    for x, _ in pbar:
        x = x.to(device, non_blocking=True)

        # t_long for indexing schedules, t for model time-embedding
        t_long = torch.randint(0, timesteps, (x.size(0),), device=device)
        t = t_long.float() / timesteps

        with torch.cuda.amp.autocast():
            x_t, eps = perturb_input(x, t_long) # ONLY ONCE
            eps_pred = nn_model(x_t, t)         # model gets float t
            loss = mse(eps_pred, eps)

        optim.zero_grad(set_to_none=True)
        scaler.scale(loss).backward()
        scaler.step(optim)
        scaler.update()

        # bookkeeping
        loss_val = float(loss.item())
        losses_step.append(loss_val)

        running += loss_val
        n_steps += 1

    pbar.set_description(f"loss: {loss_val:.4f}")

    epoch_mean = running / max(n_steps, 1)
    losses_epoch.append(epoch_mean)
    print(f"epoch {ep} mean loss: {epoch_mean:.4f}")

```

عکس 5. حلقه آموزش

```

@torch.no_grad()
def sample_ddpm(n_sample: int):
    ema_model.eval()
    x = torch.randn(n_sample, 1, height, height, device=device)

    for t_ in reversed(range(timesteps)):
        t_long = torch.full((n_sample,), t_, device=device, dtype=torch.long)
        t = (t_long.float() / timesteps).unsqueeze(1) # (B,1)

        eps = ema_model(x, t)

        beta = extract(b_t, t_long, x.shape)
        alpha = extract(a_t, t_long, x.shape)
        ab = extract(ab_t, t_long, x.shape)

        # x0 prediction + clip
        x0_hat = (x - torch.sqrt(1.0 - ab) * eps) / torch.sqrt(ab)
        x0_hat = x0_hat.clamp(-1.0, 1.0)

        # mean using clipped x0_hat (DDPM posterior mean in x0-param form)
        mean = (1.0 / torch.sqrt(alpha)) * (x - (beta / torch.sqrt(1.0 - ab)) * eps)

        if t_ == 0:
            x = mean
        else:
            var = extract(posterior_var, t_long, x.shape)
            var = torch.clamp(var, min=1e-20)
            x = mean + torch.sqrt(var) * torch.randn_like(x)

    return x

```

عکس 6. پیاده سازی نمونه برداری DDPM

```

@torch.no_grad()
def sample_ddim(n_sample: int, n_steps: int = 20):
    nn_model.eval()

    x = torch.randn(n_sample, 1, height, height, device=device)

    step_size = timesteps // n_steps
    t_seq = list(range(timesteps - 1, -1, -step_size))
    if t_seq[-1] != 0:
        t_seq.append(0)

    for i in range(len(t_seq) - 1):
        t_cur = t_seq[i]
        t_next = t_seq[i + 1]

        t_long = torch.full((n_sample,), t_cur, device=device, dtype=torch.long)
        t = (t_long.float() / timesteps).unsqueeze(1) # (B,1) float, matches training

        eps_theta = nn_model(x, t)

        alpha_bar_cur = extract(alphas, torch.full((n_sample,), t_cur, device=device, dtype=torch.long), x.shape)
        alpha_bar_next = extract(alphas, torch.full((n_sample,), t_next, device=device, dtype=torch.long), x.shape)

        # x0 prediction
        x0_hat = (x - torch.sqrt(1.0 - alpha_bar_cur) * eps_theta) / torch.sqrt(alpha_bar_cur)

        # DDIM deterministic update (eta=0)
        x = torch.sqrt(alpha_bar_next) * x0_hat + torch.sqrt(1.0 - alpha_bar_next) * eps_theta

    return x

```

عکس 7. پیاده سازی نمونه برداری DDIM



عکس 8. 50 نمونه تولید شده توسط DDPM



عکس 9. 50 نمونه تولید شده توسط DDIM

در ادامه هم مقایسه‌ای از زمان اجرای دو مدل DDIM و DDPM و همچنین FID score آن‌ها را مشاهده می‌کنیم. همانگونه که انتظار می‌رود زمان اجرای DDIM به مراتب کمتر است اما امتیاز FID آن هم بدتر (بیشتر) است و DDPM در این زمینه بهتر است. همچنین باید توجه داشت که FID score برای 500 نمونه تصادفی یکسان محاسبه شده است.

```
%timeit -r 1 sample_ddim(50, 80)
%timeit -r 1 sample_ddpm(50)
```

1.4 s $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)
16.8 s $\pm$ 0 ns per loop (mean $\pm$ std. dev. of 1 run, 1 loop each)

عکس 10. مقایسه زمان اجرا

```
FID: real: 100%|██████████| 500/500 [00:07<00:00, 63.33img/s]
FID: fake (DDIM): 100%|██████████| 500/500 [00:18<00:00, 27.65img/s]
FID: real: 100%|██████████| 500/500 [00:04<00:00, 116.86img/s]
FID: fake (DDPM): 100%|██████████| 500/500 [02:47<00:00, 2.98img/s]
FID (DDIM, n=500): 76.0763
FID (DDPM, n=500): 55.5885
Better (lower is better): DDPM
```

عکس 11. مقایسه FID score

زیر بخش دوم)

مرحله اول)

```
# STUDENT TODO: Update these prompts for your specific subject
instance_prompt = "a photo of a fantasy book" # Use a unique token like 'sks'
class_prompt = "a photo of a book" # Generic class prompt
validation_prompt = "a photo of a fantasy book in a bucket"
```

عکس 12. پرامپت های انتخابی

Image 1



Image 2

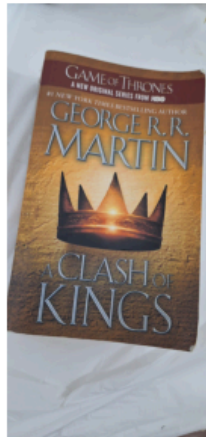


Image 3

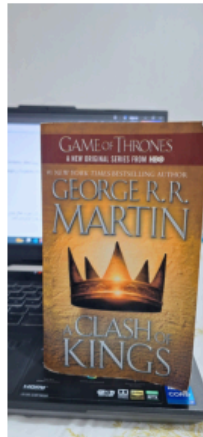


Image 4

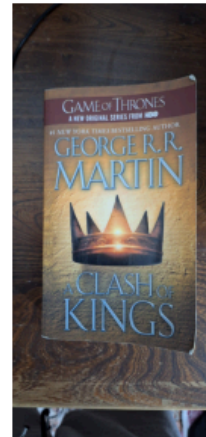
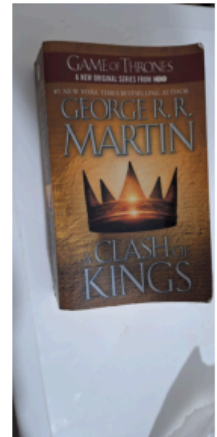


Image 5



عکس 13. پنج عکس انتخابی

(مرحله دوم)

```
def __getitem__(self, index):
    example = {}

    instance_image_path = self.instance_images_path[index % self.num_instance_images]
    instance_image = Image.open(instance_image_path)
    instance_image = exif_transpose(instance_image)
    if instance_image.mode != "RGB":
        instance_image = instance_image.convert("RGB")

    example["instance_images"] = self.image_transforms(instance_image)

    if self.encoder_hidden_states is not None:
        # (Optional) pre-computed text embeddings
        example["instance_prompt_ids"] = self.encoder_hidden_states
    else:
        text_inputs = tokenize_prompt(self.tokenizer, self.instance_prompt, self.tokenizer_max_length)
        example["instance_prompt_ids"] = text_inputs.input_ids
        if "attention_mask" in text_inputs:
            example["instance_attention_mask"] = text_inputs.attention_mask

    if self.class_data_root:
        class_image_path = self.class_images_path[index % self.num_class_images]
        class_image = Image.open(class_image_path)
        class_image = exif_transpose(class_image)
        if class_image.mode != "RGB":
            class_image = class_image.convert("RGB")

        example["class_images"] = self.image_transforms(class_image)

    if self.class_prompt_encoder_hidden_states is not None:
        example["class_prompt_ids"] = self.class_prompt_encoder_hidden_states
    else:
        class_text_inputs = tokenize_prompt(self.tokenizer, self.class_prompt, self.tokenizer_max_length)
        example["class_prompt_ids"] = class_text_inputs.input_ids
        if "attention_mask" in class_text_inputs:
            example["class_attention_mask"] = class_text_inputs.attention_mask

    return example
```

عكس 14. متد __getitem__

(مرحله سوم)

```
def generate_class_images(args, device=None):
    device = device or ("cuda" if torch.cuda.is_available() else "cpu")
    class_dir = Path(args.class_data_dir)
    class_dir.mkdir(parents=True, exist_ok=True)

    existing = []
    for ext in ("*.png", "*.jpg", "*.jpeg", "*.webp"):
        existing += list(class_dir.glob(ext))
    num_existing = len(existing)

    if num_existing >= args.num_class_images:
        print(f"Found {num_existing} class images. No need to generate more.")
        return

    num_to_gen = args.num_class_images - num_existing
    print(f"Generating {num_to_gen} class images to '{class_dir}' using prompt: {args.class_prompt!r}")

    torch_dtype = torch.float16 if (device == "cuda" and args.mixed_precision in ["fp16", "bf16"]) else torch.float32
    pipe = StableDiffusionPipeline.from_pretrained(
        args.pretrained_model_name_or_path,
        torch_dtype=torch_dtype,
        safety_checker=None, feature_extractor=None,
    )
    pipe.scheduler = DPMSolverMultistepScheduler.from_config(pipe.scheduler.config)
    pipe = pipe.to(device)
    pipe.set_progress_bar_config(disable=True)

    generator = torch.Generator(device=device).manual_seed(args.seed)

    start_idx = num_existing
    for i in tqdm(range(num_to_gen), desc="Class image gen"):
        with torch.autocast(device_type="cuda", dtype=torch.float16, enabled=(device == "cuda" and torch_dtype == torch.float16)):
            image = pipe(
                prompt=args.class_prompt,
                num_inference_steps=30,
                guidance_scale=7.5,
                generator=generator,
            ).images[0]

            out_path = class_dir / f"class_{start_idx + i:04d}.png"
            image.save(out_path)

    del pipe
    if device == "cuda":
        torch.cuda.empty_cache()
    print("Class image generation done.")

# Run if prior preservation enabled
if args.with_prior_preservation:
    generate_class_images(args)
else:
    print("Prior preservation is disabled; skipping class image generation.")
```

عكس 15. توليد تصاوير عمومي


```

global_step = 0

progress_bar = tqdm(range(args.max_train_steps), disable=not accelerator.is_local_main_process)
progress_bar.set_description("Steps")

for epoch in range(args.num_train_epochs):
    unet.train()
    if args.train_text_encoder:
        text_encoder.train()

    for step, batch in enumerate(train_dataloader):
        with accelerator.accumulate(unet):
            pixel_values = batch["pixel_values"].to(dtype=weight_dtype)

            if vae is not None:
                model_input = vae.encode(pixel_values).latent_dist.sample()
                model_input = model_input * vae.config.scaling_factor
            else:
                model_input = pixel_values

            bsz = model_input.shape[0]
            timesteps = torch.randint(
                0, noise_scheduler.config.num_train_timesteps, (bsz,), device=model_input.device
            )
            timesteps = timesteps.long()

            encoder_hidden_states = encode_prompt(
                text_encoder,
                batch["input_ids"],
                batch.get("attention_mask", None),
                text_encoder_use_attention_mask=args.text_encoder_use_attention_mask,
            )

            noise = torch.randn_like(model_input)

            noisy_model_input = noise_scheduler.add_noise(model_input, noise, timesteps)

            model_pred = unet(noisy_model_input, timesteps, encoder_hidden_states).sample

            if noise_scheduler.config.prediction_type == "epsilon":
                target = noise
            elif noise_scheduler.config.prediction_type == "v_prediction":
                target = noise_scheduler.get_velocity(model_input, noise, timesteps)
            else:
                raise ValueError(f"Unknown prediction_type: {noise_scheduler.config.prediction_type}")

            if args.with_prior_preservation:
                model_pred, model_pred_prior = model_pred.chunk(2, dim=0)
                target, target_prior = target.chunk(2, dim=0)

                loss_instance = F.mse_loss(model_pred.float(), target.float(), reduction="mean")
                loss_prior = F.mse_loss(model_pred_prior.float(), target_prior.float(), reduction="mean")
                loss = loss_instance + args.prior_loss_weight * loss_prior
            else:
                loss = F.mse_loss(model_pred.float(), target.float(), reduction="mean")

            accelerator.backward(loss)
            if accelerator.sync_gradients:
                accelerator.clip_grad_norm_(unet.parameters(), args.max_grad_norm)
            optimizer.step()
            lr_scheduler.step()
            optimizer.zero_grad()

        if accelerator.sync_gradients:
            progress_bar.update(1)
            global_step += 1

            if global_step % args.checkpointing_steps == 0:
                save_path = os.path.join(args.output_dir, f"checkpoint-{global_step}")
                accelerator.save_state(save_path)

    if global_step >= args.max_train_steps:
        break

```

عکس 16. حلقه آموزش

مرحله پنجم)



عکس 17. عکس تولیدی اول

این عکس با پرامپت [a photo of a fantasy book in a bucket](#) تولید شده



عکس 18. عکس تولیدی دوم

این عکس با پرامپت [a photo of a fantasy book at the beach](#) تولید شده



عکس 19. عکس تولیدی سوم

این عکس با پرآمپت a photo of a fantasy book in the mouth of a dog تولید شده



عکس 20. عکس تولیدی چهارم

این عکس با پرامپت [a photo of a fantasy book in a shelf](#) تولید شده



عکس 21. عکس تولیدی پنجم

این عکس با پرامپت [a photo of a fantasy book in a bookstore](#) تولید شده

سؤال دوم

بخش اول

(سوال اول)

الف) در Flow Matching به جای فرآیند تصادفی رو به جلو/عقب، یک دینامیک پیوسته تعریف می‌کنیم که نمونه‌ها را در زمان حرکت می‌دهد:

$$v_{\theta}(x, t) = \frac{dx}{dt}$$

اگر در زمان $t=0$ از توزیع ساده (مثلاً نویز گاوسی) نمونه بگیریم:

$$x(0) \sim P_0(\text{noise})$$

و معادله‌ی بالا را تا $t=1$ حل کنیم، به نمونه‌ی نهایی می‌رسیم:

$$x(1) = \phi_1(x(0)) \sim P_1$$

که ϕ_t «جریان» (Flow) القا شده توسط میدان v_θ است. این میدان برداری در واقع سرعت/جهت حرکت ذره را در هر نقطه از فضا و هر زمان تعیین می‌کند؛ بنابراین با یادگیری v_θ ، داریم «جرم احتمال» را از P_0 به سمت P_{data} جابه‌جا می‌کنیم.

ب) تفاوت ها: **Diffusion**: معمولاً یک فرآیند تصادفی تعریف می‌کند (زنجیره‌ی مارکوف یا SDE) و سپس فرآیند معکوس (حذف نویز) را یاد می‌گیرد؛ نمونه‌برداری غالباً گام‌های زیاد و (اغلب) stochastic دارد.

Flow Matching: یک **ODE قطعی** با میدان $v_\theta(x, t)$ یاد می‌گیرد؛ نمونه‌برداری یعنی حل عددی ODE (اغلب پایدارتر/کم‌مرحله‌تر).

در Diffusion معمولاً «اسکور/نویز» یاد گرفته می‌شود (مثل ϵ_θ یا $s_\theta = \nabla_x \log P_t(x)$).

در Flow Matching مستقیماً «سرعت/میدان برداری» یاد گرفته می‌شود: $v_\theta(x, t)$.

در بسیاری از Flow/Continuous Normalizing Flow ها برای likelihood دقیق باید ترم‌های واگرایی/ژاکوبین وارد شود؛ اما Flow Matching معمولاً با یک رگرسیون مستقیم برای میدان (بدون محاسبه‌ی صریح ژاکوبین) آموزش داده می‌شود.

در Diffusion، آموزش رایج از نوع score matching / denoising (مثل MSE نویز) است.

شباهت ها: وقتی diffusion را در فرم پیوسته در نظر بگیریم و به **Probability Flow ODE** (نسخه‌ی قطعی متناظر با SDE) برویم، دینامیک نمونه‌برداری آن هم یک ODE می‌شود؛ در این حالت، Flow Matching عملاً دارد همان نوع مسیر قطعی از نویز به داده را مدل می‌کند و رفتارشان نزدیک می‌شود (همچنین در نمونه‌برداری‌های قطعی diffusion مثل حالت‌های deterministic).

سوال دوم

الف) یک انتخاب رایج این است:

$$x_t = (1 - t)x_0 + tx_1, t \in [0, 1]$$

در این روش مسیر ساده و قابل کنترل است و مشتق زمانی هم ساده است.

$$\frac{dx_t}{dt} = x_1 - x_0$$

یعنی «سرعت هدف» روی این مسیر (برای یک جفت (x_0, x_1) یک بردار ساده/ثابت می‌شود؛ پس آموزش به یک رگرسیون ساده‌تر تبدیل می‌شود.

این مسیر همیشه بهینه نیست، اگر توزیع داده پیچیده/چندمودی باشد، خط مستقیم بین نویز و داده ممکن است از نواحی کم‌چگالی عبور کند و میدان برداری لازم برای تصحیح مسیر پیچیده‌تر شود. مسیرهای بهتر می‌توانند غیرخطی باشند (مثلاً مسیرهای نزدیک‌تر به Optimal Transport یا مسیرهایی که خمیدگی/انحراف کمتر و سازگارتر با هندسه‌ی داده دارند).

(ب) با مسیر خطی، هدف آموزشی معمولاً ساده‌تر می‌شود چون x_t و مشتق آن فرم بسته دارند؛ در نتیجه loss اغلب یک MSE رگرسیون مستقیم بین خروجی مدل و سرعت هدف است. این سادگی معمولاً باعث گرادیان‌های پایدارتر، نیاز کمتر به ترفندهای پیچیده و پیاده‌سازی آسان‌تر می‌شود؛ هرچند اگر مسیر خطی از نظر هندسی بد باشد، ممکن است برای مدل سخت‌تر شود و ظرفیت/گام‌های حل‌گر بیشتری لازم شود.

سوال سوم

در score matching شرطی سعی می‌کنیم $\nabla_{x_t} \log q(x_t|x_0)$ پس زیان ما در کل می‌شود:

$$E_{t, x_0 \sim P_{data}, x_t \sim q(\cdot|x_0)} [\|s_\theta(x_t, t) - \nabla_{x_t} \log q(x_t|x_0)\|_2^2]$$

همچنین می‌دانیم که تابع امتیاز توزیع نرمال همان نمای‌اش است با باز پارامتردهی می‌توانیم آن را اینطور بنویسیم $x_t = x_0 + \sigma_t \epsilon$ که اپسیلون یک توزیع نرمال استاندارد است. پس در کل داریم

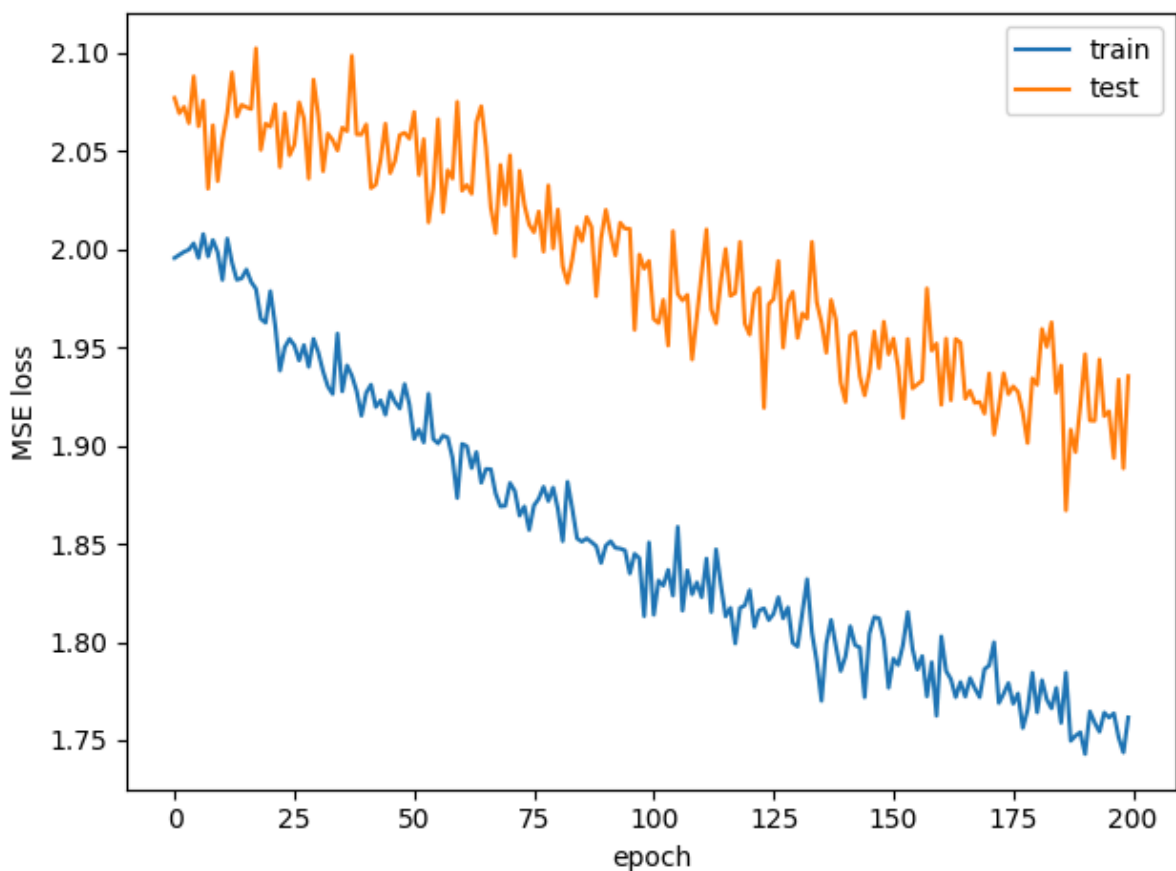
$$E_{t, x_0 \sim P_{data}, x_t \sim q(\cdot|x_0)} [\|s_\theta(x_t, t) - \frac{x_0 - x_t}{\sigma_t^2}\|_2^2]$$

بخش دوم

مرحله اول)

این گام انجام شد ولی از آنجایی که برای دویست epoch اجرا شده خیلی عکسش بزرگ خواهد شد پس در گزارش قرارش نمی دهیم ولی در نوتبوک و همچنین در مرحله بعدی نمودارش را نمایش می دهیم.

مرحله دوم)



عکس 22. زیان آموزش و تست

همانگونه که مشاهده می شود فاصله بین این دو خیلی نیست و هر دو نزولی هستند اما زیان آزمون کمی بهتر است و در انتها فاصله بین آنها نسبت به آغاز بیشتر می شود و این نشانه‌ای از کمی بیش برآزش است. کم بودن داده ها و همچنین همبستگی بین آن ها به دلیل سری زمانی بودن از علل این اتفاق است.

```

@torch.no_grad()
def sample_ode_euler(model: nn.Module, n_samples: int, seq_len: int, n_steps: int):
    model.eval()
    x = torch.randn(n_samples, seq_len, device=device)
    dt = 1.0 / n_steps
    for k in range(n_steps):
        t = torch.full((n_samples, 1), k * dt, device=device)
        v = model(x, t)
        x = x + dt * v
    return x

@torch.no_grad()
def sample_ode_heun(model: nn.Module, n_samples: int, seq_len: int, n_steps: int):
    model.eval()
    temprature = 0.8
    x = torch.randn(n_samples, seq_len, device=device) * temprature
    dt = 1.0 / n_steps
    for k in range(n_steps):
        t0 = torch.full((n_samples, 1), k * dt, device=device)
        v0 = model(x, t0)
        x_pred = x + dt * v0
        t1 = torch.full((n_samples, 1), (k + 1) * dt, device=device)
        v1 = model(x_pred, t1)
        x = x + 0.5 * dt * (v0 + v1)
    return x

# تولید داده مصنوعی
if SOLVER.lower() == "euler":
    gen_norm_t = sample_ode_euler(model, N_GEN, SEQ_LEN, ODE_STEPS)
else:
    gen_norm_t = sample_ode_heun(model, N_GEN, SEQ_LEN, ODE_STEPS)

gen_norm = gen_norm_t.detach().cpu().numpy().astype(np.float32)
gen_denorm = gen_norm * std_train + mean_train # بازگشت به مقیاس اصلی log-return

print("Generated:", gen_norm.shape, gen_denorm.shape)

```

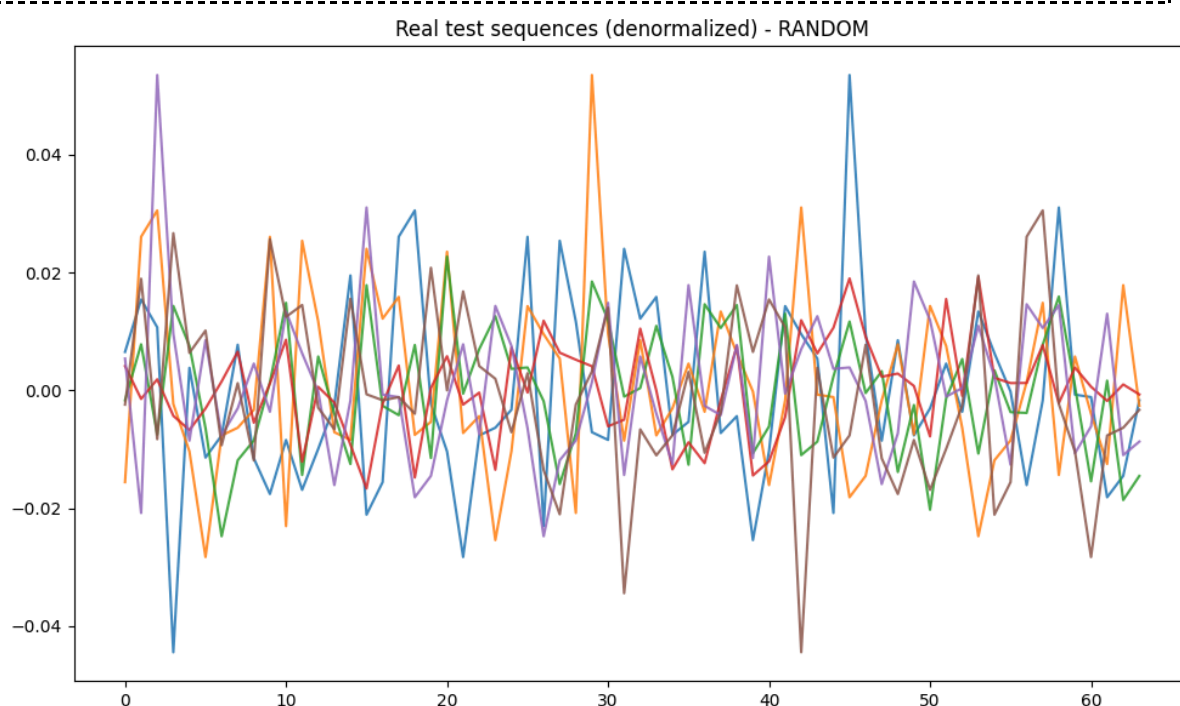
عکس 23. منطق تولید نمونه

در این روش با یک نویز گوسی استاندارد که 64 بعدی است شروع می کنیم و در هر مرحله این الگوریتم یک پیش بینی اوپلر انجام می دهیم: $x_{pred} = x_k + \Delta t v_{\theta}(x_k, t_k)$ است. سپس شیب را در نقطه ی پیش بینی شده محاسبه کرده و میانگین دو شیب را به عنوان به روزرسانی نهایی به کار می بریم: $x_{k+1} = x_k + \frac{\Delta t}{2} (v_{\theta}(x_k, t_k) + v_{\theta}(x_{pred}, t_{k+1}))$ همچنین Δt معکوس N است که N در این مورد 64 است. با افزایش N، حل عددی به حل دقیق ODE نزدیک تر می شود و خطای انتگرال گیری کاهش می یابد؛ بنابراین معمولاً کیفیت نمونه ها بهتر می شود (به خصوص اگر میدان برداری به خوبی یاد گرفته شده باشد). هزینه محاسباتی تقریباً خطی با تعداد گام ها رشد می کند، چون در هر گام باید شبکه v_{θ} یک یا چند بار اجرا شود. برای روش Heun در هر گام تقریباً ۲ بار

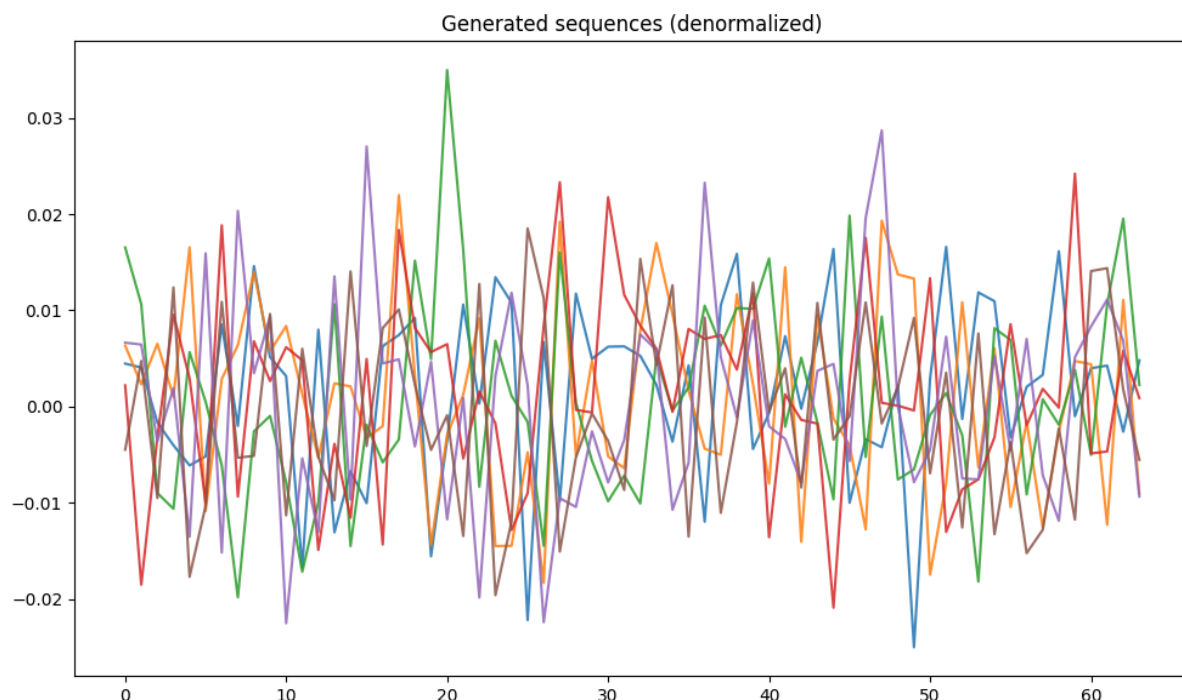
forward لازم است (محاسبه‌ی v_0 و v_1)، پس: $cost \propto 2N$ بنابراین دو برابر کردن تعداد گام‌ها تقریباً دو برابر زمان نمونه‌گیری را افزایش می‌دهد. با کاهش N ، خطای عددی بیشتر می‌شود و مسیر تولید می‌تواند از مسیر واقعی منحرف شود؛ در نتیجه ممکن است نمونه‌ها نویزی‌تر یا دارای اعوجاج توزیعی شوند.

نکته‌ی تجربی مهم: اگر خود میدان برداری v_0 دقیق یاد گرفته نشده باشد، افزایش N لزوماً کیفیت را خیلی بهتر نمی‌کند، چون محدودیت اصلی از مدل است نه از حل‌گر.

مرحله چهارم)



عکس 24. نمونه واقعی



عکس 25. نمونه تولید شده توسط مدل

محور افقی: اندیس زمانی داخل پنجره‌ی ۶۴ تایی (۰ تا ۶۳)، یعنی روزهای متوالی داخل هر سکانس.
محور عمودی: مقدار log-return روزانه در مقیاس اصلی (Denormalized).
هر خط رنگی: یک سکانس مستقل ۶۴ تایی (رنگ‌ها فقط برای تفکیک بصری هستند و معنی خاصی ندارند).

در نمودار واقعی، مقادیر بازده معمولاً حول صفر قرار دارند و گهگاه جهش‌های بزرگ تا حدود $\pm 4\%$ تا $\pm 5\%$ دیده می‌شود.

در نمودار تولیدی نیز بازده‌ها حول صفر هستند و دامنه‌ی نوسان عمدتاً در حدود $\pm 2\%$ تا $\pm 3\%$ است و چند جهش محدودتر دیده می‌شود.

نتیجه: دامنه‌ی کلی تولیدی‌ها با داده واقعی سازگار است (حول صفر و با اندازه نوسان مشابه)، هرچند در این نمونه‌های خاص، داده‌ی واقعی چند جهش بزرگ‌تر هم نشان می‌دهد و تولیدی‌ها کمی ملایم‌تر به نظر می‌رسند.

هر دو نمودار رفتار نویزگونه و بدون روند مشخص دارند.
 شدت نوسان در تولیدی‌ها به شکل کلی با واقعی‌ها هم‌مقیاس است (جهت و اندازه تغییرات روزانه مشابه است).

نتیجه: از نظر نوسان، سری‌های تولیدی در مقیاس اصلی به مقدار واقعی نزدیک هستند و رفتار کلی بازده روزانه را بازتولید می‌کنند.

توالی‌های واقعی و تولیدی هر دو فاقد روند بلندمدت درون پنجره‌ی ۶۴ روزه هستند و حول صفر نوسان می‌کنند (رفتار مورد انتظار برای returns). در داده واقعی گاهی خوشه‌های نوسان (چند روز پرنوسان پشت سر هم) کمی بیشتر دیده می‌شود؛ در داده تولیدی این اثر ضعیف‌تر به چشم می‌آید.

نتیجه: شکل کلی تولیدی‌ها از نظر ماهیت سری بازده (بدون روند و حول صفر) سازگار است، اما شدت/پایداری خوشه‌بندی نوسان در داده واقعی ممکن است کمی بیشتر باشد.

البته شایان ذکر است که سری‌های تولیدی در ابتدا خیلی خوب نبودند اما با اضافه کردن temperature به منطق نمونه‌گیری واریانس نمونه‌های تولیدی خیلی به نمونه‌های واقعی نزدیک شد ولی میانگین‌ها کمی دور شدند ولی قبل از این کار میانگین‌ها تقریباً منطبق بودند ولی واریانس‌ها 30% فاصله داشتند!

مرحله پنجم

این تحلیل را بر اساس دو تصویر قبلی انجام می‌دهم.

۱) حفظ الگوهای کلی سری زمانی

رفتار کلی بازده‌ها در هر دو نمودار مشابه است: سری‌ها حول صفر نوسان می‌کنند و روند مشخصی در پنجره‌های ۶۴ روزه دیده نمی‌شود. شکل کلی توالی‌ها در هر دو حالت نویز گونه و با تغییرات روزانه نسبتاً سریع است که با ماهیت بازده‌های مالی سازگار است.

نتیجه: مدل در بازتولید ویژگی‌های کلی سری زمانی بازده (mean نزدیک صفر، نبود روند، نوسان‌های روزانه) موفق بوده است.

۲) بازتولید جهش‌ها و تغییرات شدید

در نمودار واقعی، چند جهش بزرگ‌تر (حرکت‌های شدید مثبت یا منفی) دیده می‌شود؛ یعنی گاهی بازده‌ها به شکل ناگهانی چند درصد تغییر می‌کنند. در نمودار تولیدی، جهش‌ها وجود دارند اما در نمونه‌های نمایش داده‌شده معمولاً کمتر و ملایم‌تر از

جهش‌های بزرگ در داده واقعی به نظر می‌رسند (شدت شوک‌ها کمتر است یا کمتر تکرار می‌شود).

نتیجه: مدل به صورت کلی توانسته وجود تغییرات ناگهانی را بازتولید کند، اما در بازتولید رفتارهای بسیار شدید/نادر (extreme jumps) احتمالاً کمی ضعف دارد و خروجی‌ها تا حدی صاف‌تر از واقعی دیده می‌شوند.

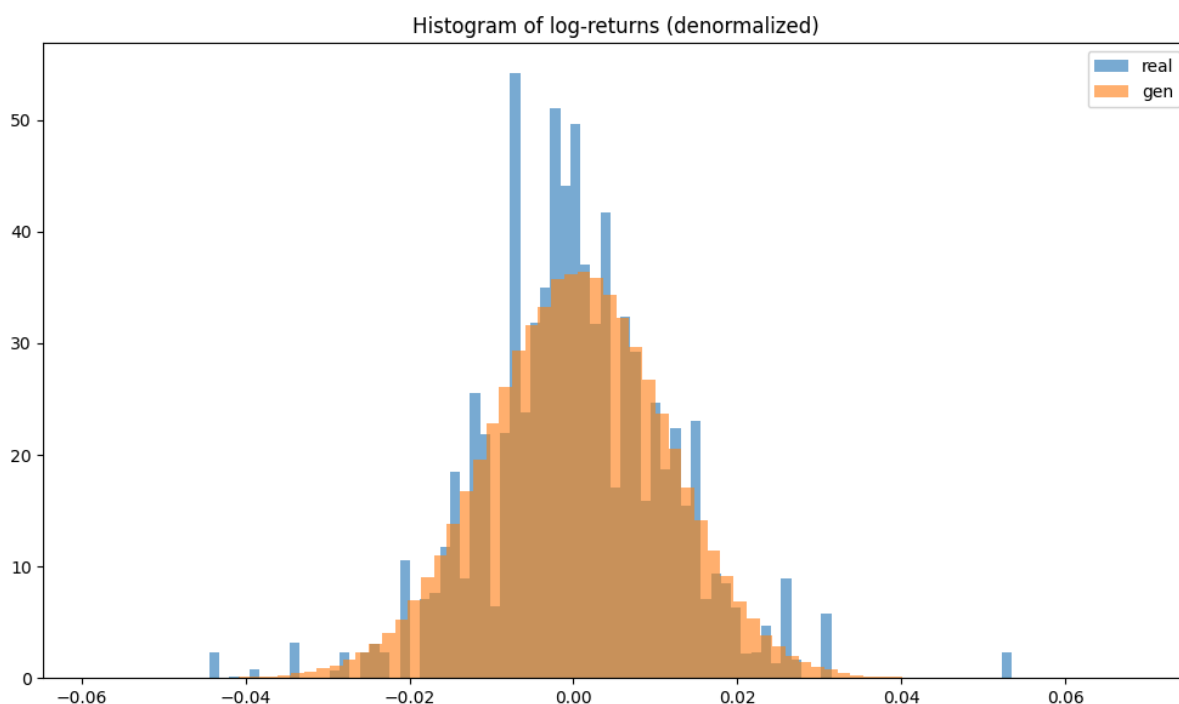
۳) الگوهای خاص مثل خوشه‌بندی نوسان (Volatility Clustering)

در داده واقعی گاهی می‌توان دوره‌هایی دید که چند روز پشت سر هم نوسان بزرگ‌تر است (خوشه‌بندی نوسان).

در داده تولیدی این اثر به صورت واضح‌تر کمتر دیده می‌شود و بسیاری از توالی‌ها نوسانی نسبتاً یکنواخت دارند.

نتیجه: مدل احتمالاً در بازتولید کامل ساختارهای زمانی نوسان (مثل خوشه‌بندی نوسان) نسبت به تطبیق توزیع کلی بازده‌ها ضعیف‌تر است.

مرحله ششم



عکس 26. هیستوگرام log-returns

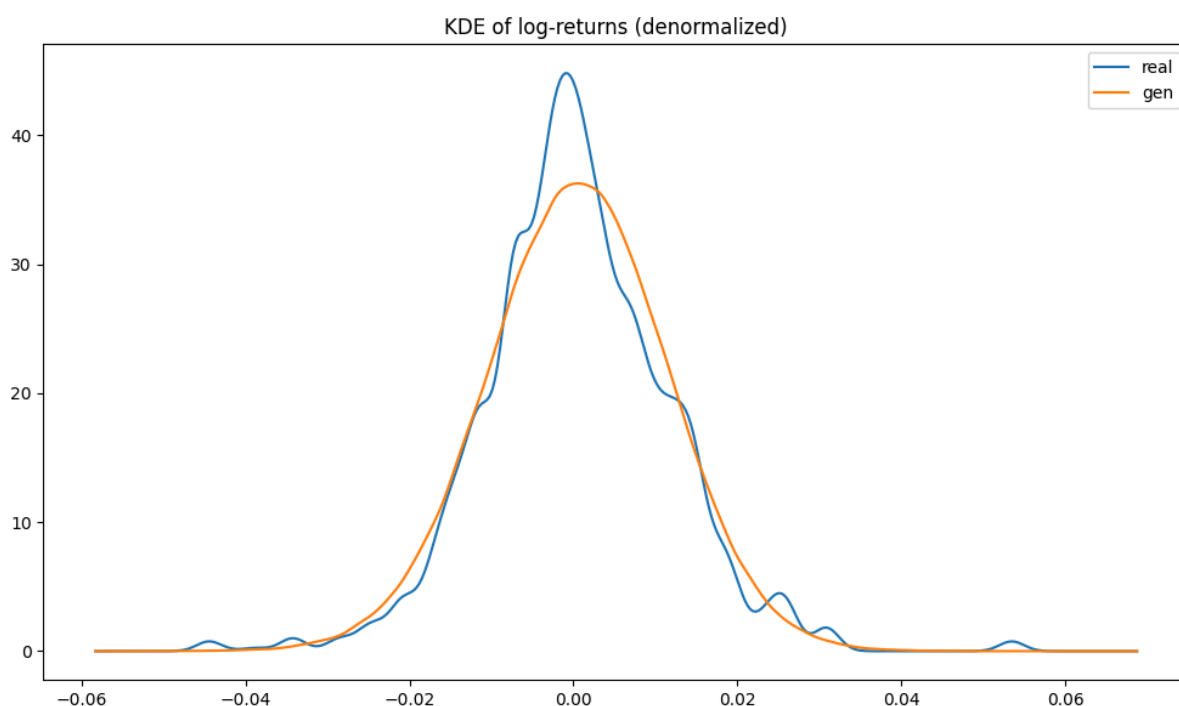
در هیستوگرام (بالا):

- توزیع‌های واقعی و تولیدی در بازه‌ی مرکزی (حدوداً بین -0.02 تا $+0.02$) به شدت روی هم افتاده‌اند، یعنی مدل بخش عمده‌ی توزیع را درست یاد گرفته است.

- در بازه‌های دورتر از صفر (دم‌ها)، در داده واقعی چند ستون پراکنده‌ی دورتر دیده می‌شود (نمونه‌های نادر با جهش‌های شدیدتر)، در حالی که توزیع تولیدی در این نواحی معمولاً صاف‌تر و کمتر پرشی است.

این رفتار طبیعی است چون:

- دم‌ها در داده مالی کم‌نمونه هستند و تخمین آن‌ها سخت‌تر است؛
- Histogram نسبت به KDE حساسیت بیشتری به تعداد نمونه‌ها و binning دارد، بنابراین اختلاف‌های کوچک در دم‌ها در هیستوگرام بیشتر به چشم می‌آید.



عکس 27. KDE of log-returns

در نمودار KDE (بالا)، هر دو منحنی شکل زنگوله‌ای و تقریباً متقارن حول صفر دارند که نشان می‌دهد:

- مرکز توزیع در هر دو مجموعه نزدیک به صفر است $\Leftarrow$ مدل میانگین/Drift را به خوبی بازتولید کرده است.
- منحنی تولیدی (gen) از نظر پهنا و محل قله با منحنی واقعی (real) تقریباً همپوشانی زیادی دارد $\Leftarrow$ واریانس کلی و مقیاس نوسان تا حد زیادی درست بازتولید شده است.
- یک تفاوت قابل مشاهده این است که قله‌ی KDE واقعی کمی تیزتر/بلندتر است و منحنی gen کمی پهن‌تر به نظر می‌رسد؛ این می‌تواند نشان دهد که داده واقعی کمی بیشتر روی مقادیر خیلی نزدیک صفر متمرکز است و داده تولیدی کمی پراکندگی بیشتری دارد. با این حال اختلاف کوچک است.

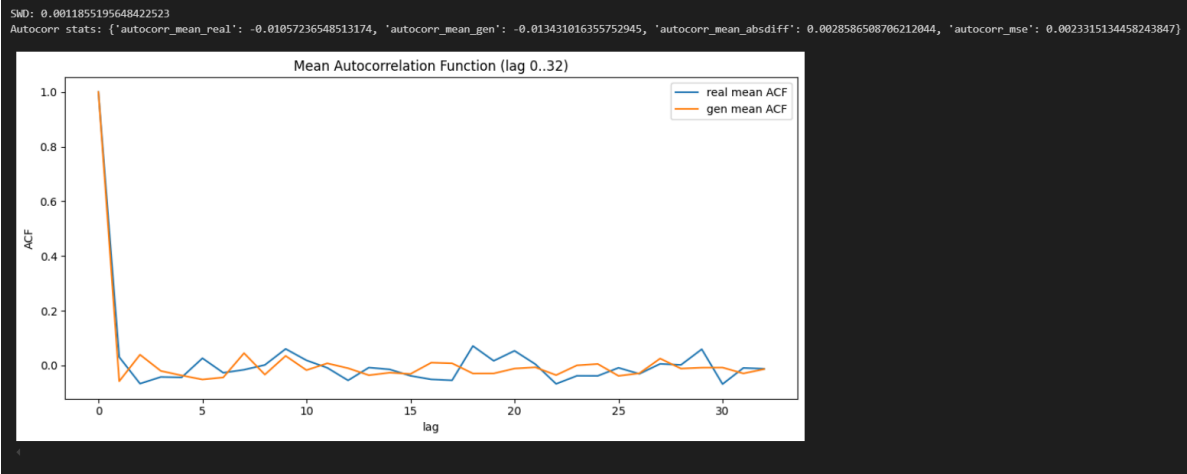
مرحله هفتم)

```
Real: mean=0.000400 var=0.000128 vol=0.011293
Gen : mean=0.000462 var=0.000122 vol=0.011059
```

عکس 28. مقایسه آماری توزیع مدل و واقعی

میانگین‌ها بسیار نزدیکند و واریانس و نوسان هم همینطور منتهی دو تای آخر در مدل کمترند بر خلاف میانگین. با توجه به اینکه نوسان تولیدی تنها حدود ۲٪ کمتر از مقدار واقعی است، می‌توان گفت مدل شدت نوسانات بازار را تا حد زیادی درست بازتولید کرده است و اختلاف مشاهده شده کوچک و قابل قبول است.

در عین حال، این اختلاف نشان می‌دهد مدل ممکن است در بازتولید دقیق‌ترین رخدادهای پرنوسان (جهش‌های خیلی شدید) کمی محافظه‌کارانه‌تر عمل کند، اما در مقیاس کلی نوسان روزانه، تطابق خوبی با داده‌ی واقعی دارد.



عکس 29. فاصله Wasserstein و معیار autocorrelation

الف) SWD چه چیزی را اندازه می‌گیرد؟ توالی‌های ۶۴ تایی را می‌توان به صورت بردارهای ۶۴ بعدی دید. فاصله‌ی Wasserstein (فاصله‌ی انتقال جرم) میزان تفاوت بین دو توزیع را اندازه می‌گیرد، ولی محاسبه‌ی آن در بعد بالا سخت است. SWD این مشکل را با ایده‌ی زیر حل می‌کند: داده‌های واقعی و تولیدی را روی تعداد زیادی جهت تصادفی (projection) به ۱ بعد نگاشت می‌کند، فاصله‌ی Wasserstein یک‌بعدی را روی هر projection حساب می‌کند، و میانگین می‌گیرد.

پس SWD نشان می‌دهد توزیع توالی‌های تولیدی در فضای چند بعدی چقدر به توزیع توالی‌های واقعی نزدیک است (شامل شکل کلی، پهنای توزیع، و تا حدی روابط بین ابعاد).

SWD = 0.00119 مقدار نسبتاً کوچک است، یعنی توزیع چندبعدی توالی‌های تولیدی به توزیع واقعی نزدیک شده است. با توجه به اینکه در مراحل قبل mean/std و کوانتایل‌ها هم نزدیک بودند، این SWD پایین تأیید می‌کند که مدل از نظر شباهت توزیعی عملکرد خوبی دارد.

ب) برای هر توالی زمانی، تابع اتوکورلیشن (ACF) میزان همبستگی سری با نسخه‌ی شیفت‌خورده‌ی خودش را در lagهای مختلف اندازه می‌گیرد.

- **Autocorrelation Mean:** میانگین مقدار ACF (معمولاً روی lag های 1 تا 32) یعنی یک عدد خلاصه از "میزان وابستگی زمانی کلی" (اگر نزدیک صفر باشد یعنی سری تقریباً بی‌وابستگی است).

• **Autocorrelation MSE**: خطای میانگین مربعات بین ACF واقعی و تولیدی در lagهای مختلف (lag 1..32)

یعنی بررسی می‌کند که شکل منحنی ACF (وابستگی در هر lag) چقدر بین real و gen مشابه است.

تحلیل کیفی و نتیجه‌گیری

1. میانگین ACF هر دو نزدیک صفر است

این برای بازده‌های روزانه طبیعی است (returns معمولاً autocorrelation زیادی ندارند). پس مدل توانسته ویژگی تقریباً بدون وابستگی زمانی بودن بازده‌ها را تا حد زیادی حفظ کند.

2. اختلاف میانگین ACF خیلی کوچک است (≈ 0.0029)

یعنی از نظر کلی، میزان وابستگی زمانی تولیدی‌ها تقریباً مشابه واقعی‌هاست.

3. **Autocorrelation MSE = 0.00233**

این مقدار نشان می‌دهد بین شکل دقیق منحنی ACF واقعی و تولیدی تفاوت‌هایی وجود دارد، اما اختلاف آن خیلی بزرگ نیست. در نمودار ACF هم دیده می‌شود که هر دو منحنی بعد از $lag=0$ عمدتاً حول صفر هستند، ولی در بعضی lag real نوسان‌های کوچک مثبت/منفی دارد و gen صاف‌تر یا کمی متفاوت رفتار می‌کند.

از نظر خود بازده‌ها (returns) مدل وابستگی‌های زمانی را تا حد خوبی یاد گرفته است. چون هم ACF تولیدی و هم واقعی نزدیک صفرند و اختلاف‌ها کوچک است. یعنی مدل علاوه بر شباهت توزیعی، ویژگی کم‌بودن وابستگی زمانی بازده‌ها را نیز بازتولید کرده است.

مراجع